

Webmap Deployment Guide

This repository is fully containerized with **Docker** and **Docker Compose**, making it incredibly easy to deploy on any Virtual Private Server (VPS) like DigitalOcean, Hetzner, AWS, etc.

Prerequisites for the Server

The server must have **Docker** and **Docker Compose** installed. (If using a fresh Ubuntu server, install them via: `sudo apt update && sudo apt install docker.io docker-compose -y`)

How to Deploy

1. **Clone or Copy this repository** onto your server.
2. Navigate into the **Webmap** directory:

```
cd Webmap
```

3. **Update your Database Credentials in .env (Important!)** A unified credential system is in place. You must store your secure credentials in the **.env** file in the root directory.

(If the **.env** file did not copy over to the server, create it by running `cp .env.example .env`)

Open the file to edit:

```
nano .env
```

Modify it to look like this (replace with your secure passwords):

```
POSTGRES_USER=your_secure_user
POSTGRES_PASSWORD=your_secure_password
POSTGRES_DB=webmap
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
```

(Note: The Python backend automatically reads this **.env** file. When running via Docker Compose, Docker intelligently overrides **POSTGRES_HOST** to route internally to the database container, meaning you use the exact same **.env** file for both local development and production without changing code!)

4. **Start everything in the background:**

```
sudo docker-compose up -d --build
```

What happens when you run this?

1. **Database (db)**: Starts a `postgis` container using the credentials from your `.env` file.
2. **Backend (backend)**: Builds a Python image, installs all requirements using `uv`, waits 10 seconds for the database to boot, and automatically runs `import_local_maps.py` to seed the database with your GeoJSON maps from the `Maps/` folder. It then starts the FastAPI server on port `8484`.
3. **Frontend (frontend)**: Starts an Nginx server on port `80`. It serves the frontend static files (`html/js/css`) and automatically routes all `/api/...` requests to the Python backend container.

How to access the site

Once it finishes building and starts up, you can simply visit the server's IP address (e.g., `http://YOUR_SERVER_IP`) in your web browser.

Troubleshooting

If the maps do not show up immediately, the backend might still be processing the GeoJSON files into the database. You can check the backend logs using:

```
sudo docker-compose logs -f backend
```

If you need to stop the servers:

```
sudo docker-compose down
```